**Karel Gonzalez**

**Procesamiento Masivo de Datos**

# Implementación de WordCount con Hadoop MapReduce para el conteo de palabras de la novela El conde de Montecristo

## 1. Inicialización de Hadoop:

Para comenzar con la práctica, inicié los servicios de Apache Hadoop desde la terminal de Ubuntu mediante el comando start-hadoop. Posteriormente, verifiqué que los procesos necesarios estuvieran activos, identificando el NameNode, DataNode, SecondaryNameNode, ResourceManager y NodeManager. Estos componentes permiten administrar el almacenamiento de archivos mediante HDFS y gestionar los recursos necesarios para ejecutar trabajos MapReduce. De esta manera, comprobé que el entorno de procesamiento se encontraba disponible antes de comenzar a trabajar con los archivos.

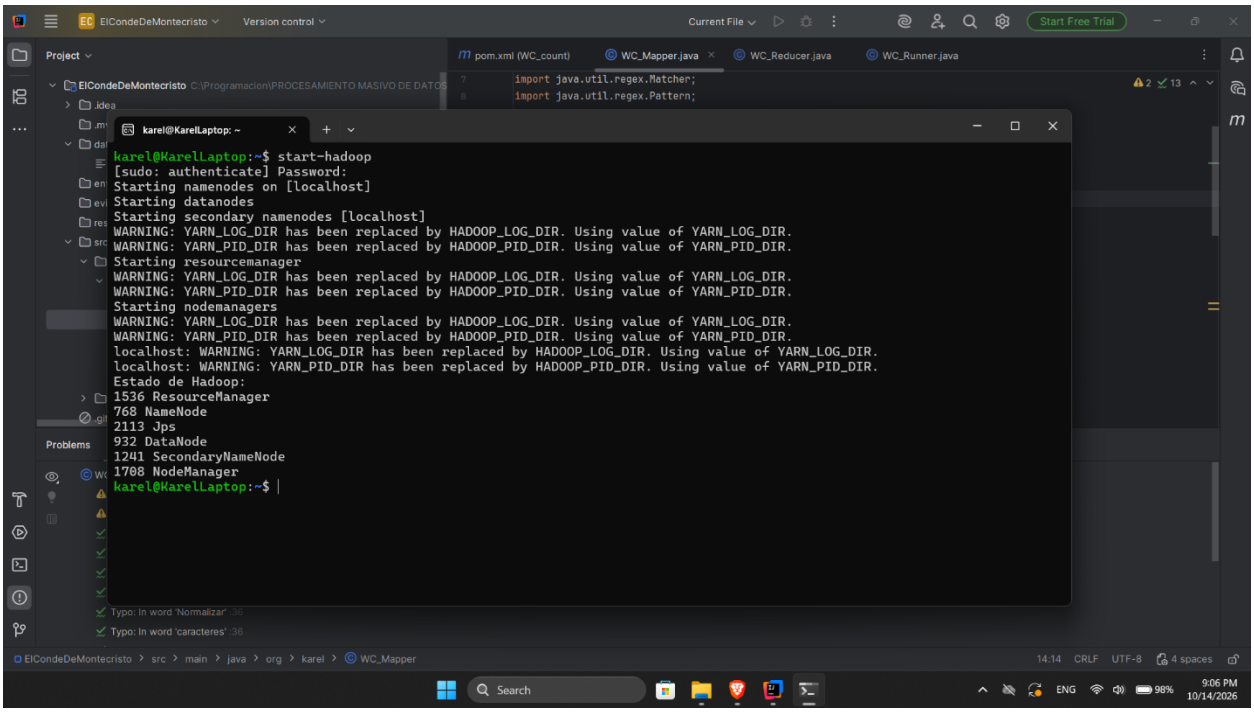



Figura 1. Inicialización de los servicios de Apache Hadoop y verificación de los procesos activos.

**2. Selección del libro y preparación del proyecto:**

Para realizar el conteo de palabras, seleccioné la novela *The Count of Monte Cristo*, de Alexandre Dumas y Auguste Maquet, disponible gratuitamente en Project Gutenberg con el identificador 1184. Descargué el libro en formato TXT y lo incorporé a la carpeta datos del proyecto desarrollado en IntelliJ IDEA. Se decidió conservar el contenido original del archivo, sin eliminar encabezados, información editorial ni otros elementos, ya que el objetivo era ejecutar WordCount sobre el documento completo. Asimismo, organicé el proyecto utilizando Maven y tres clases Java encargadas de implementar el procesamiento MapReduce.

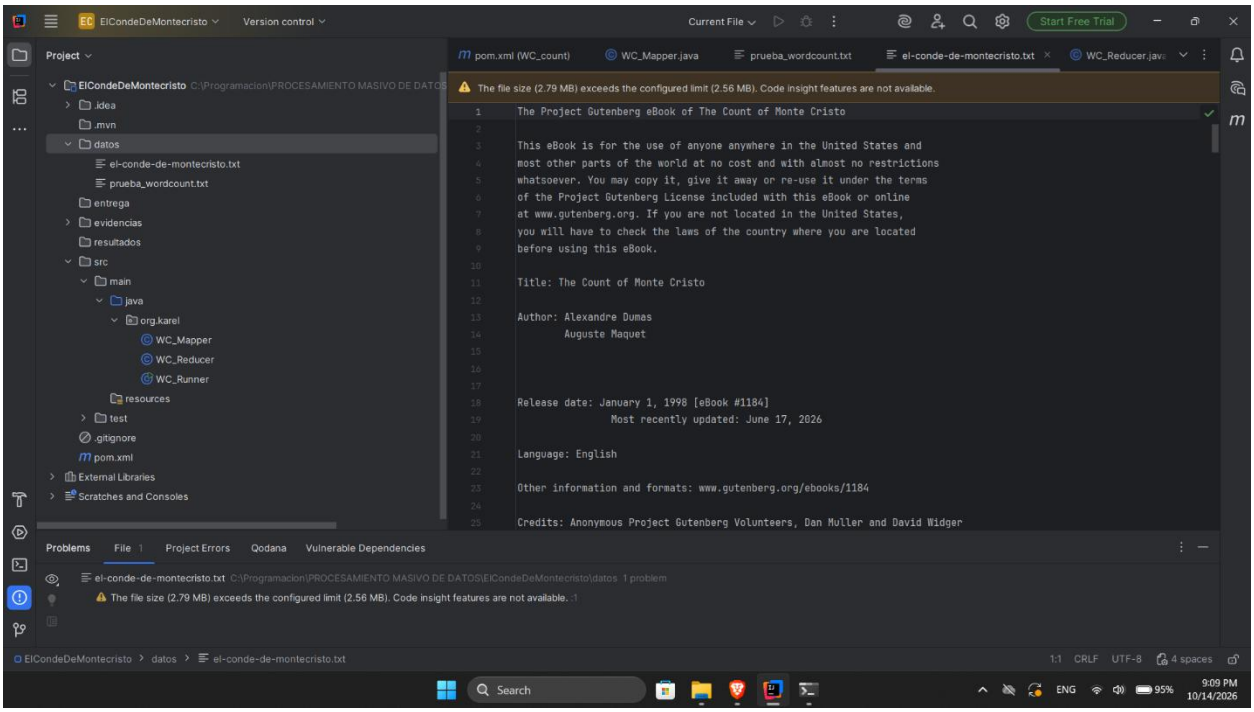



Figura 2. Estructura del proyecto en IntelliJ IDEA y archivo original de The Count of Monte Cristo.

**3. Compilación del proyecto mediante Maven:**

Una vez implementadas las clases WC_Mapper, WC_Reducer y WC_Runner, procedí a compilar el proyecto mediante Maven desde IntelliJ IDEA. Durante este procedimiento se verificaron las dependencias de Hadoop y se empaquetaron las clases desarrolladas en el archivo ejecutable WC_count-1.0-SNAPSHOT.jar, ubicado dentro del directorio target. La compilación finalizó satisfactoriamente, mostrando el mensaje BUILD SUCCESS, lo cual permitió comprobar que el código podía compilarse y empaquetarse correctamente antes de enviarlo a Hadoop para su ejecución.

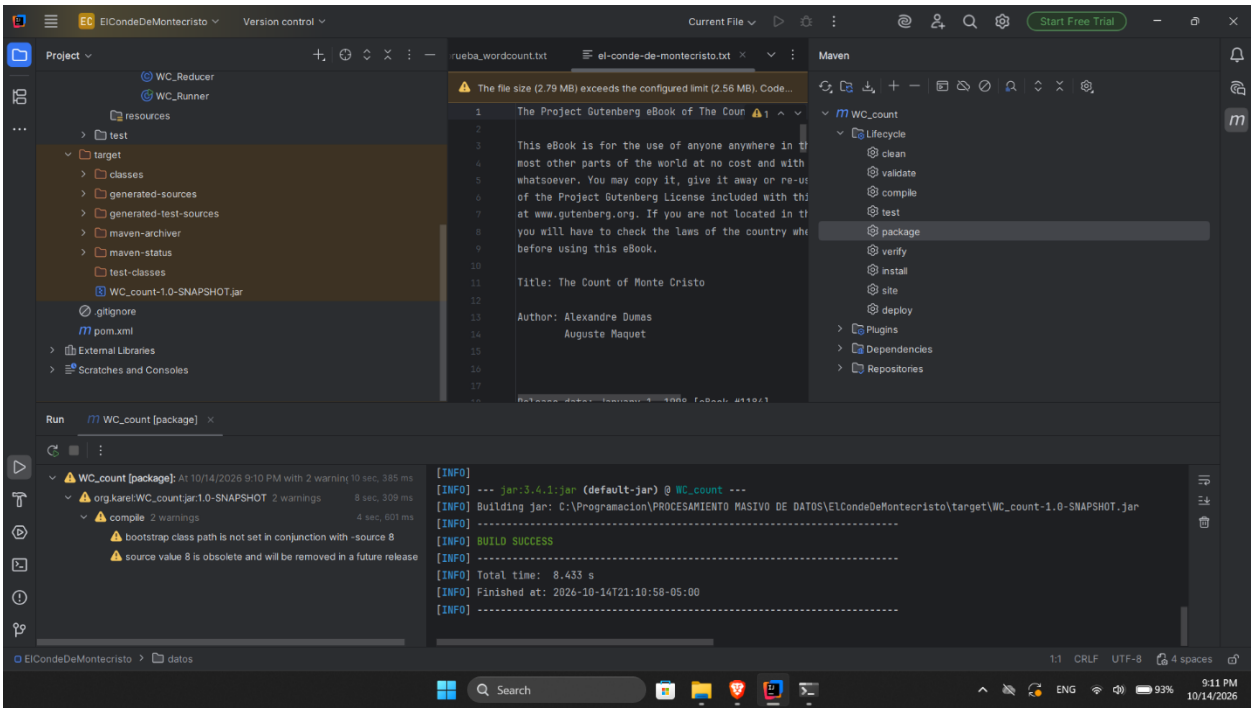



Figura 3. Compilación exitosa del proyecto Java mediante Maven y generación del archivo JAR.

**4. Ejecución de prueba del proceso WordCount:**

Antes de procesar el libro completo, realicé una ejecución de prueba utilizando un archivo TXT de menor tamaño, preparado con palabras repetidas, diferencias entre mayúsculas y minúsculas y signos de puntuación. El propósito de esta prueba fue comprobar que las clases Mapper y Reducer funcionaran de acuerdo con las reglas implementadas. Para ello, cargué el archivo en HDFS y ejecuté el trabajo MapReduce utilizando el JAR generado previamente. El procesamiento finalizó correctamente, registrando 11 apariciones de palabras y cinco términos diferentes, sin presentar errores durante la ejecución.

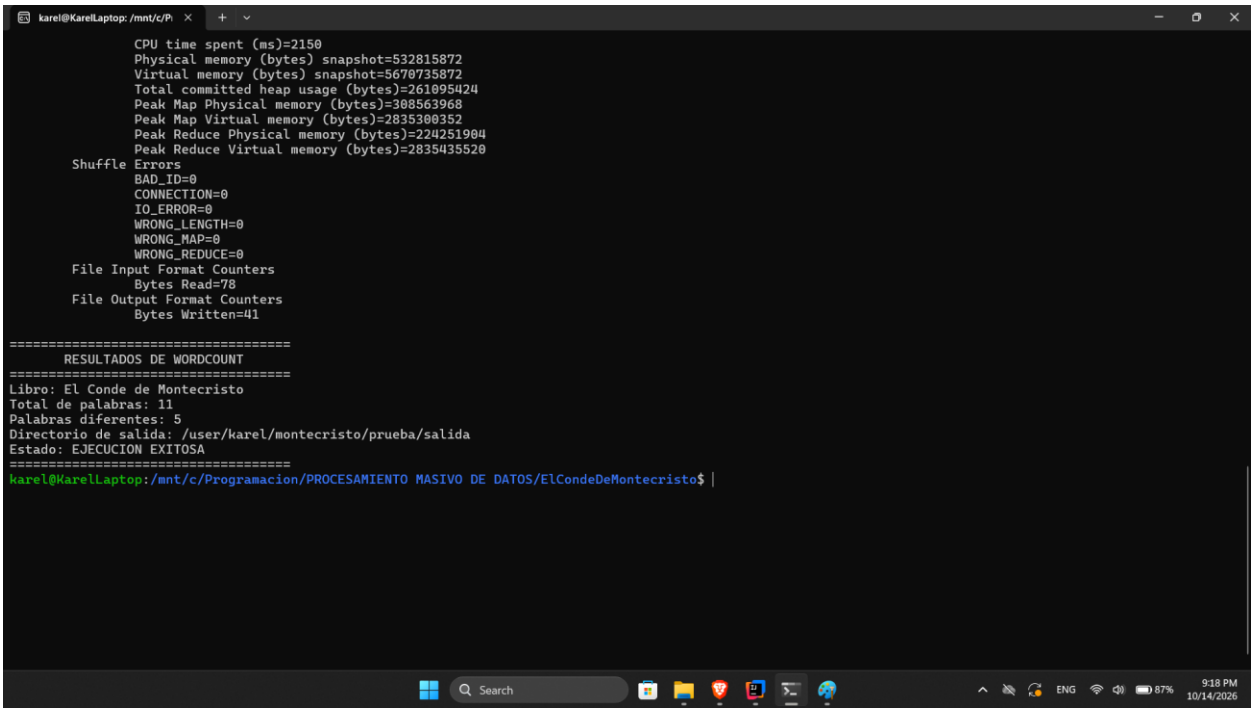



Figura 4. Ejecución exitosa de WordCount sobre el archivo de prueba almacenado en HDFS.

**5. Verificación de los resultados de prueba:**

Después de completar la ejecución de prueba, consulté el archivo generado por Hadoop mediante el comando hdfs dfs -cat, con el objetivo de verificar las frecuencias calculadas. Los resultados mostraron cinco términos diferentes: cristo, hadoop, hello, it's y monte, cuyas frecuencias fueron 2, 3, 2, 2 y 2, respectivamente. Estos valores coincidieron con el contenido del archivo de entrada, demostrando que el programa normalizaba correctamente las mayúsculas, reconocía las contracciones y acumulaba las apariciones de cada palabra. Con esta validación, se comprobó el funcionamiento básico de la implementación antes de procesar la novela.

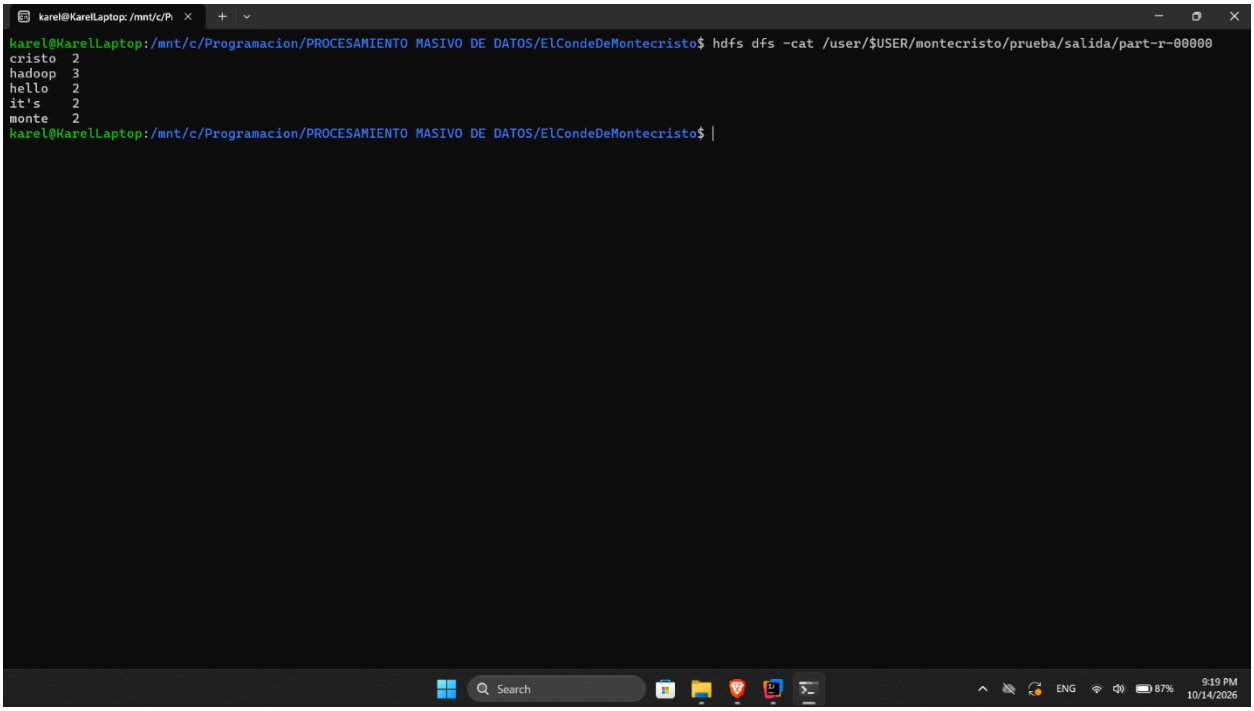



Figura 5. Verificación de las frecuencias obtenidas durante la prueba de WordCount.

**6. Carga del libro completo en HDFS:**

Una vez validado el programa, procedí a cargar el archivo original de *The Count of Monte Cristo* en el sistema de archivos distribuido de Hadoop. Primero creé el directorio de entrada mediante hdfs dfs -mkdir -p y posteriormente utilicé hdfs dfs -put para almacenar el documento dentro de la ruta /user/karel/montecristo/entrada/. Finalmente, ejecuté hdfs dfs -ls -h para comprobar que el archivo se encontrara disponible en HDFS. La verificación mostró que el libro había sido cargado correctamente, conservando su contenido original y quedando preparado para el procesamiento definitivo.

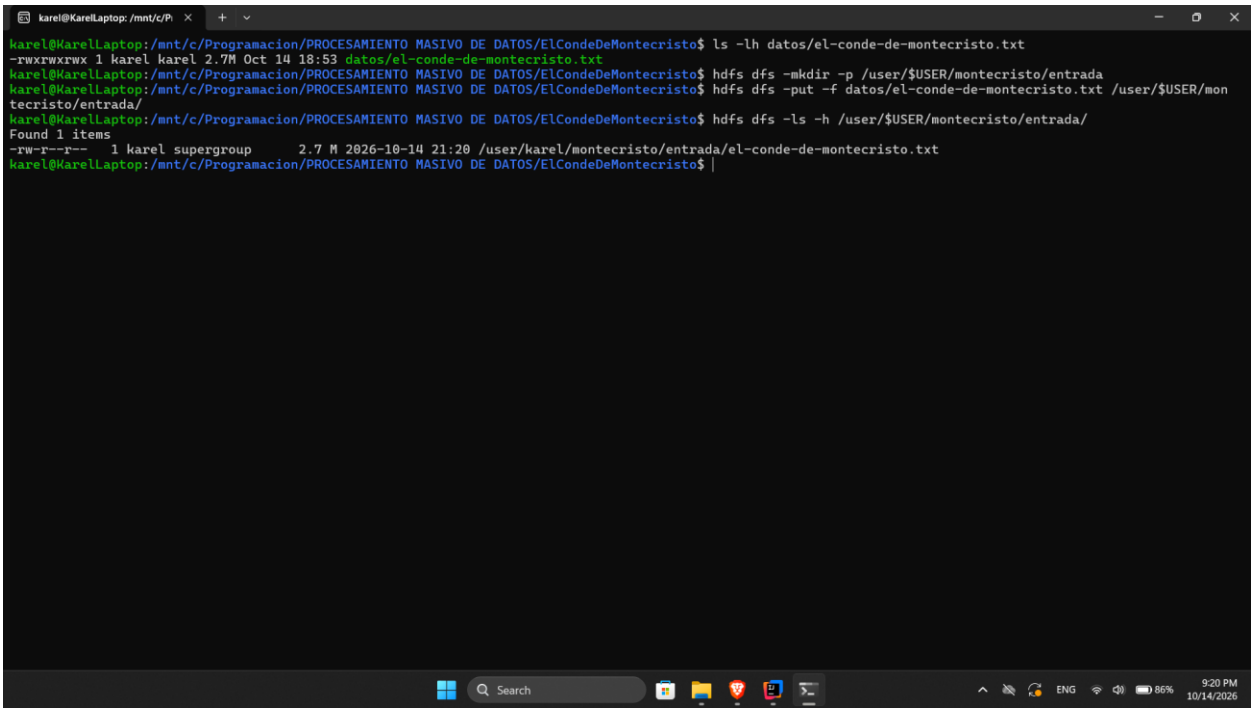



Figura 6. Carga y verificación del archivo original de The Count of Monte Cristo en HDFS.

**7. Ejecución de MapReduce sobre el libro completo:**

Con el archivo almacenado en HDFS, ejecuté el trabajo WordCount utilizando el JAR desarrollado en Java. Durante el procesamiento, el Mapper recorrió las 61.680 líneas del documento, identificando las palabras y generando los pares clave-valor correspondientes. Posteriormente, el Combiner realizó agregaciones parciales y el Reducer calculó las frecuencias definitivas de cada término. La ejecución finalizó satisfactoriamente, obteniendo un total de 467.664 apariciones de palabras y 16.242 términos diferentes. Además, los contadores de Hadoop confirmaron que no se produjeron errores de Shuffle durante el procesamiento.

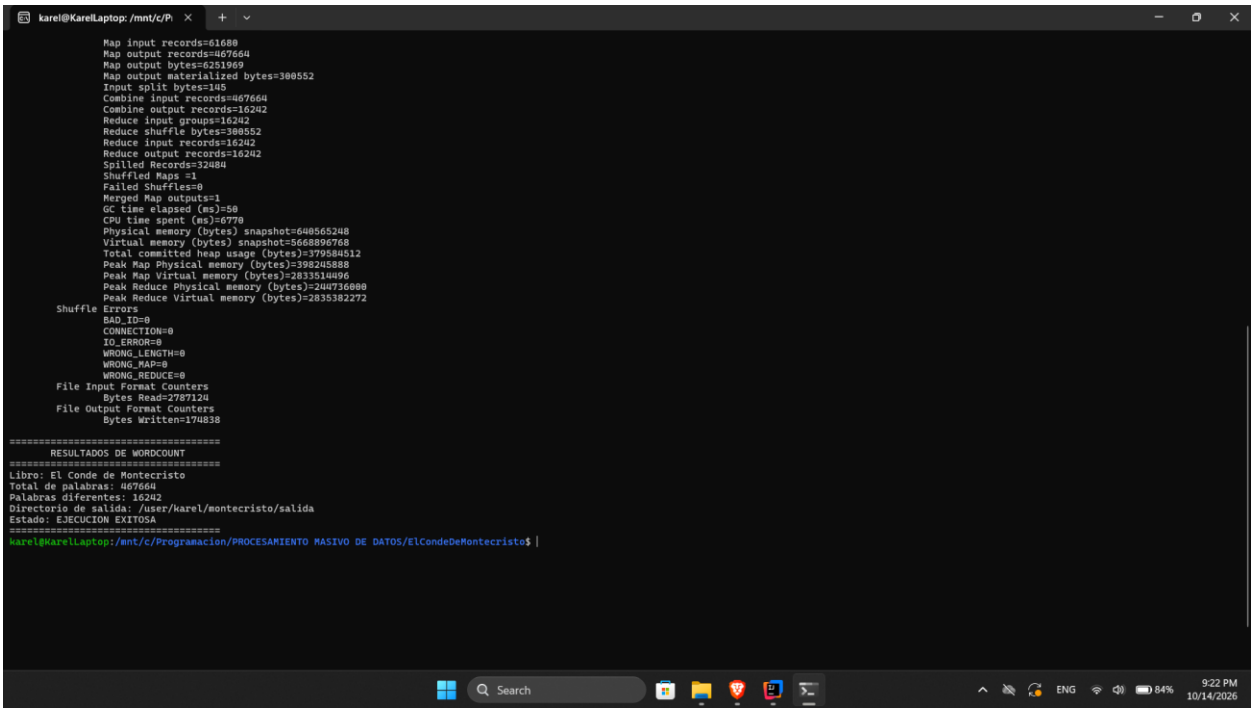



Figura 7. Ejecución exitosa de MapReduce sobre el libro completo y estadísticas del procesamiento.

**8. Verificación de los archivos de salida en HDFS:**

Después de finalizar el procesamiento, consulté el directorio de salida generado por Hadoop para comprobar que los resultados estuvieran almacenados correctamente. Mediante el comando hdfs dfs -ls -h, verifiqué la existencia de los archivos _SUCCESS y part-r-00000. El primero indica que el trabajo MapReduce finalizó satisfactoriamente, mientras que el segundo contiene las palabras identificadas junto con sus frecuencias. Posteriormente, visualicé las primeras líneas del archivo de resultados, comprobando que la información se encontraba organizada en pares de palabra y cantidad de apariciones.

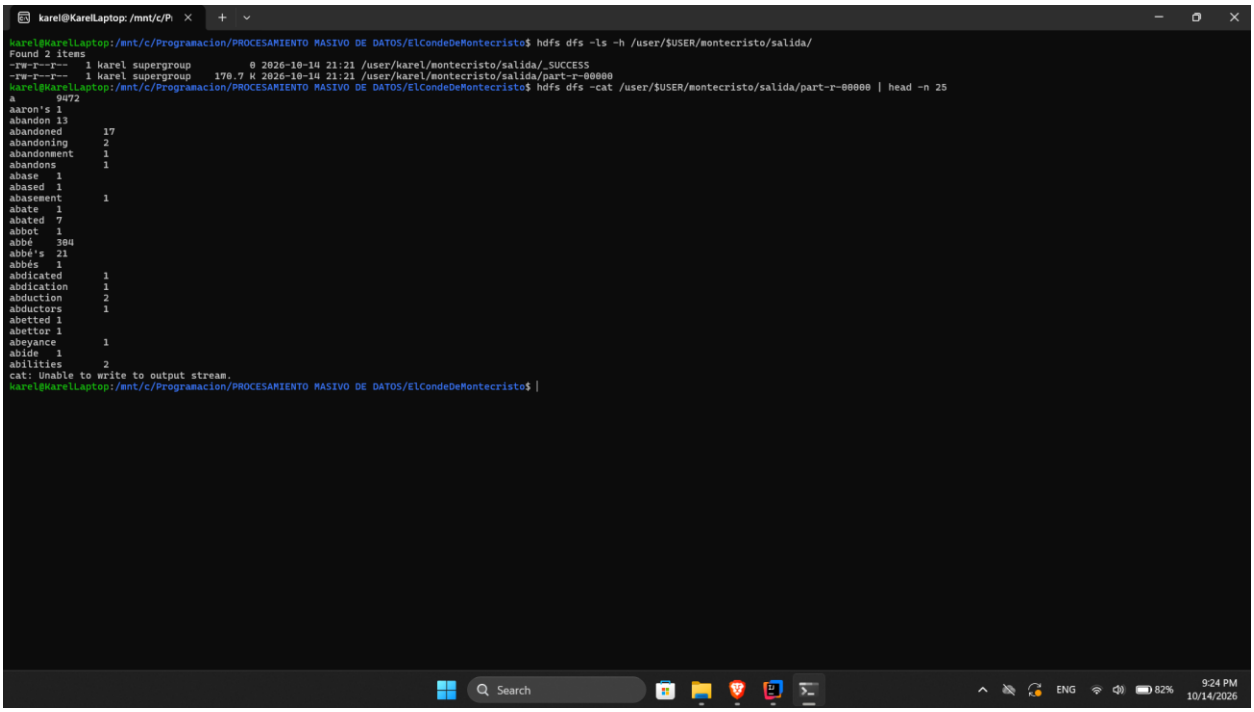



Figura 8. Verificación de los archivos generados por MapReduce y visualización del conteo almacenado en HDFS.

**9. Análisis y validación de los resultados obtenidos:**

Con el procesamiento finalizado, exporté el archivo de resultados desde HDFS al directorio local del proyecto y generé un listado con las 20 palabras más frecuentes de la novela. Entre los términos con mayor número de apariciones se encontraron palabras comunes del inglés como the, of, to y and, lo cual resulta coherente con las características del texto procesado. Adicionalmente, utilicé los comandos wc -l y awk para verificar las estadísticas obtenidas, confirmando la existencia de 16.242 términos diferentes y un total de 467.664 apariciones. Ambos valores coincidieron con los reportados por Hadoop al finalizar el trabajo.

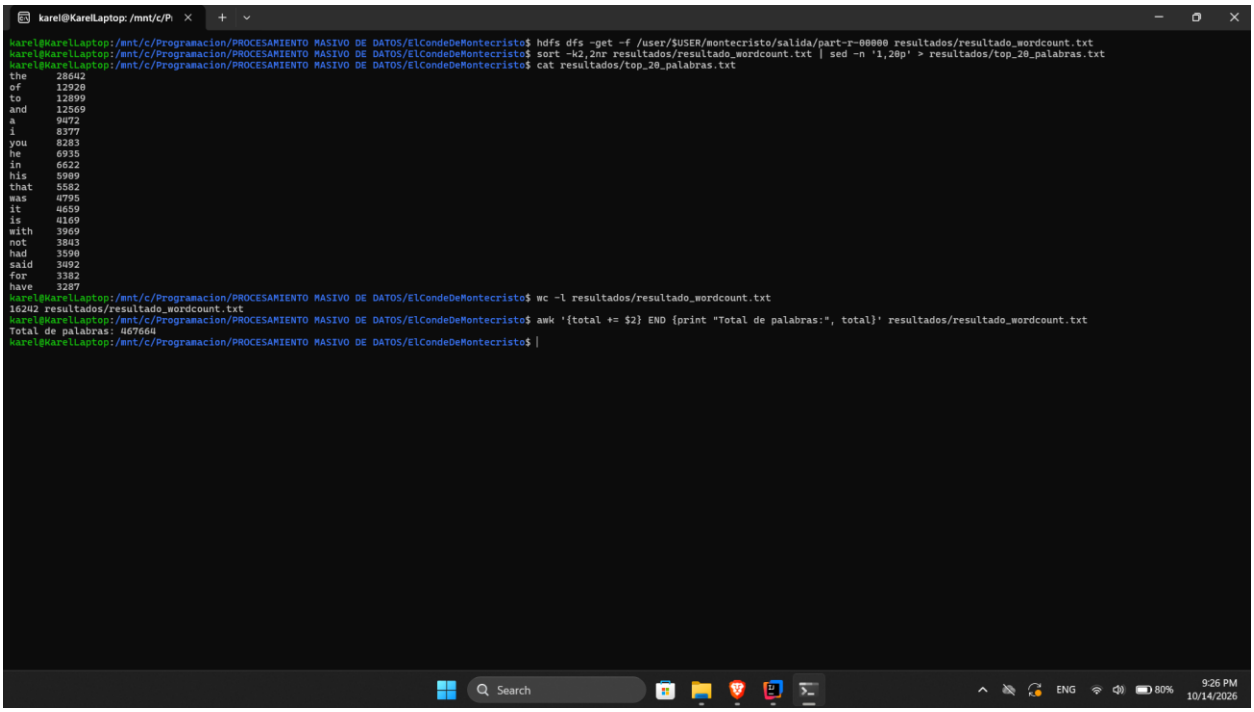



Figura 9. Identificación de las 20 palabras más frecuentes y validación de los resultados de WordCount.

**10. Exportación y organización de los archivos finales:**

Finalmente, incorporé los resultados del procesamiento a la carpeta resultados del proyecto desarrollado en IntelliJ IDEA. En ella se almacenaron los archivos resultado_wordcount.txt, que contiene el conteo completo de las palabras identificadas, y top_20_palabras.txt, que presenta los términos de mayor frecuencia. Esta organización permite conservar los resultados obtenidos y consultarlos posteriormente sin necesidad de ejecutar nuevamente el trabajo MapReduce. De esta manera, se completó el procedimiento solicitado, desde la selección y carga del libro en HDFS hasta la ejecución del procesamiento y la verificación de los archivos de salida.

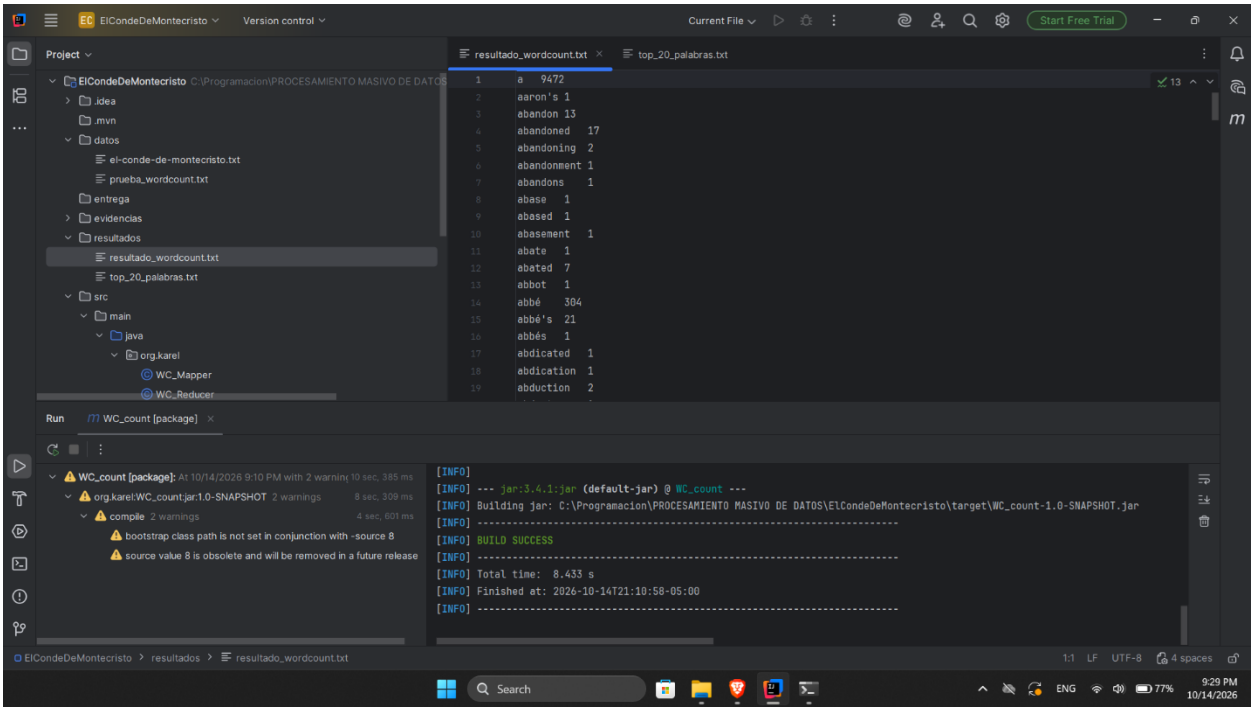



Figura 10. Organización y visualización de los archivos finales generados por WordCount en IntelliJ IDEA.

**Conclusión:**

Mediante el desarrollo de esta práctica logré implementar y ejecutar satisfactoriamente el proceso WordCount utilizando Apache Hadoop MapReduce, aplicándolo al libro *El conde de Montecristo*, obtenido de Project Gutenberg. Para ello, desarrollé las clases Mapper, Reducer y Runner en Java, compilé el proyecto mediante Maven y utilicé HDFS para almacenar tanto los archivos de entrada como los resultados generados. Antes de procesar el libro completo, realicé una prueba con un archivo de texto pequeño, lo cual me permitió verificar el funcionamiento del programa y comprobar que el conteo de palabras se realizaba correctamente. Posteriormente, procesé el documento original y obtuve un total de 467.664 apariciones de palabras y 16.242 términos diferentes, resultados que pude validar mediante comandos adicionales en Ubuntu. Esta experiencia me permitió comprender mejor cómo se relacionan los distintos componentes de Hadoop y cómo las etapas de mapeo, combinación y reducción trabajan conjuntamente para procesar información. Asimismo, pude llevar a la práctica los conocimientos adquiridos sobre procesamiento masivo de datos, desde la preparación del entorno y la ejecución de un trabajo distribuido hasta la obtención y verificación de sus resultados.

**Repositorio de GitHub**

El código fuente desarrollado en Java, los archivos de entrada, los resultados obtenidos y las evidencias correspondientes a esta práctica se encuentran disponibles en el siguiente repositorio de GitHub:

Enlace al repositorio: https://github.com/karelgonzalezruiz/wordcount-hadoop-montecristo